

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"**

**Інститут ІКНІ
Кафедра СШІ**

ЗВІТ

до лабораторної роботи №6

з дисципліни: “Середовища та компоненти розробки у моделюванні та аналізі
даних”

на тему: “Використання GitOps для автоматизованого розгортання застосунку”

Лектор:
Бенч. А.Я

Виконав:
Хомишин Д.М.

Прийняв:
Бенч. А.Я

Тема роботи: Використання GitOps для автоматизованого розгортання застосунку

Мета роботи: Навчитися використовувати GitOps для автоматизованого розгортання застосунку

РЕЗУЛЬТАТИ

1. Використані технології

1. Azure Kubernetes Service (AKS)
2. Kubernetes
3. ArgoCD
4. Docker та Docker Hub
5. GitHub
6. Helm
7. Prometheus
8. Grafana
9. Flask

2. Короткий опис проєкту

У попередніх лабораторних роботах було створено пайплайн для аналізу та візуалізації даних про якість повітря. Проєкт складався з декількох сервісів Docker Compose: `data_load`, `data_quality_analysis`, `data_research`, `visualization` та `web`. У даній лабораторній роботі було реалізовано GitOps-підхід для автоматичного розгортання web-сервісу у Kubernetes-кластері AKS.

3. Створення Kubernetes-кластера в Azure

1. Виконано авторизацію в Azure CLI за допомогою `az login`.
2. Створено Resource Group у регіоні North Europe.
3. Створено AKS-кластер із одним worker node.
4. Налаштовано `kubectl` та отримано доступ до Kubernetes-кластера.

4. Встановлення та налаштування ArgoCD

Для реалізації GitOps-підходу було використано ArgoCD. ArgoCD встановлено в окремий namespace `argocd`. Після встановлення було виконано `port-forward` та вхід у web-інтерфейс системи.

5. Підготовка Docker image

Для deployment у Kubernetes було створено Docker image Flask-застосунку. Image зібрано для платформи `linux/amd64` за допомогою Docker Buildx, оскільки AKS використовує `amd64` worker nodes.

Image було завантажено у Docker Hub repository: `khomdanylo/air-quality-web`.

6. Створення Kubernetes manifests

Було створено Kubernetes manifests для namespace, deployment та service. Deployment описував Flask web-застосунок, а Service типу LoadBalancer надавав публічний доступ до застосунку через Azure Load Balancer.

7. Реалізація GitOps deployment

Усі Kubernetes manifests було збережено у GitHub repository. ArgoCD використовував GitHub як source of truth та автоматично синхронізував стан Kubernetes-кластера із manifests у репозиторії.

Після зміни Docker image tag та виконання git push ArgoCD автоматично оновлював deployment у кластері без ручного втручання.

8. Реалізація rollback

Було продемонстровано rollback deployment шляхом повернення попередньої версії Docker image у deployment manifest. Після git push ArgoCD автоматично виконав rollback до попередньої версії застосунку.

9. Моніторинг Kubernetes-кластера

Для моніторингу Kubernetes-кластера було встановлено kube-prometheus-stack через Helm. Stack включав Prometheus, Grafana, kube-state-metrics та node-exporter.

У Grafana було використано готові Kubernetes dashboards, які дозволяли переглядати CPU, memory та network metrics для namespace air-quality та pod-ів застосунку.

ВИСНОВКИ

У ході виконання лабораторної роботи було успішно реалізовано GitOps-підхід для автоматизованого deployment застосунку у Kubernetes-середовищі Azure AKS. Було отримано практичні навички роботи з Kubernetes, ArgoCD, Docker Hub, Prometheus та Grafana. Реалізоване рішення дозволяє автоматично синхронізувати стан кластера з GitHub repository, виконувати rollback deployment та здійснювати моніторинг стану Kubernetes-кластера і застосунку в режимі реального часу.